

LLMSched: 面向复合 LLM 应用的不确定性感知工作负载调度

Botao Zhu¹, Chen Chen², Xiaoyi Fan³, Yifei Zhu^{1*}

¹UM-SJTU Joint Institute, Shanghai Jiao Tong University, Shanghai, China

²John Hopcroft Center for Computer Science, Shanghai Jiao Tong University, Shanghai, China

³Jiangxing Intelligence Inc., Shenzhen, China

Email: {zhubotao, chen-chen}@sjtu.edu.cn, xiaoyi.fan@ieee.org, yifei.zhu@sjtu.edu.cn

摘要—开发复合大语言模型 (LLM) 应用, 正日益成为解决现实问题的常见方法。在这类应用中, LLM 与多种外部模块协作, 包括 API, 甚至其他 LLM, 以实现复杂的智能服务。然而, 我们发现, 复合 LLM 应用固有的时长不确定性和结构不确定性, 给 LLM 服务提供方的高效服务与调度带来重大挑战。本文提出 LLMSched, 一个面向新兴复合 LLM 应用的不确定性感知调度框架。在 LLMSched 中, 我们首先设计一种新的基于 DAG 的模型, 用于描述不确定的复合 LLM 应用; 随后采用贝叶斯网络, 对复合 LLM 应用进行全面性能分析, 并识别能够降低不确定性的阶段, 同时设计基于熵的机制, 量化这些阶段带来的不确定性降低。通过结合不确定性降低策略与高效优化作业完成时间 (JCT) 的方案, 我们进一步提出高效调度器, 以降低平均 JCT。针对多种代表性复合 LLM 应用的仿真和测试平台实验表明, 与现有最先进调度方案相比, LLMSched 能将平均 JCT 降低 14~79%。

关键词—大语言模型, DAG 调度, 资源管理, 复合 LLM

I. 引言

大语言模型 (LLM) 正在革新数字世界, 并成为通用人工智能 (AGI) 的关键组成部分。拥有数十亿参数的 LLM 在海量人类语料上训练和微调, 展现出执行内容摘要、代码生成和机器翻译等 NLP 任务的卓越能力, 其水平能够媲美甚至超过人类专家。典型 LLM 包括 GPT-4 [1] 和 Gemini [2] 等强大的闭源模型, 以及 Llama [3]、Vicuna [4] 和 Qwen [5] 等开源模型。因此, 这些模型受到广泛欢迎, 并获得产业界和学术界的高度研究关注。例如, 作为当今代表性 LLM 的 GPT-4 [1] 拥有约 1.805 亿用户, 其中每周活跃用户达 1 亿 [6]。

为了处理更复杂的现实问题, 现有 LLM 应用已经从依赖单一整体 LLM, 转向复合 LLM 系统; 在这样的系统中, LLM 与 API 乃至其他 LLM 等多种外部模块协作, 提供复杂智能服务 [7] [8] [9]。例如, 在任务自动化应用 [10] 中, LLM 充当规划器, 为用户选择深度学习模型来解决复杂任务。为了服务这些应用, LLM 服务提供方需要将大量推理请求调度到配备 GPU 的集群上, 并实现较短响应时间。与典型 LLM 请求相比, 服务复合 LLM 应用更具挑战性, 因为它们可能包含若干乃至数十个 LLM 推理任务 [8] [11], 以及工具调用等其他非 LLM 任务。

对于由多个相互依赖阶段构成的复杂作业, 每个阶段包含一个或多个并行执行任务, 通常可以将作业表示为有向无环图 (DAG), 其中阶段对应节点, 边表示输入输出依赖。复合 LLM 应用同样可以建模为 DAG。例如, 程序合成应用 [12] 包含多次迭代的代码执行和代码生成阶段, 组织为链式 DAG。由于复合 LLM 应用固有的依赖关系, 人

们自然会考虑使用传统集群服务框架, 并应用现有 DAG 调度方案来服务这些应用。

然而, 基于覆盖数千个查询、针对三类常见复合 LLM 应用的全面实验, 我们识别出它们与数据库查询 [13] 和回归分析 [14] 等现有数据处理作业之间的两个关键差异。首先, 由于 LLM 以自回归方式生成回答 [1] [15], 复合 LLM 应用中 LLM 查询的持续时间高度不确定且难以预测 [16]。我们发现, 一个复合 LLM 应用的作业¹持续时间可从数秒到数百秒不等。LLM 推理中的批处理特征 [17] 进一步加剧了时长不确定性。其次, 相当一部分复合 LLM 应用将 LLM 作为智能体, 要求它进行规划 [10] [18], 或者在不固定的轮数内推理 [9], 导致其结构并不确定。例如, 代码生成应用 [9] 的链长度可能在 3 到 15 步之间变化。关于这两种不确定性的更多细节, 见第 III 节。

这两个独特特征给复合 LLM 应用调度带来很大挑战。具体而言, 现有 DAG 作业调度方案通常依赖充分、准确的作业时长和 DAG 结构信息 [13] [19] [20]。然而, 高度不确定的作业时长导致估计不准确, 进而降低最短作业优先 (SJF) [21] 等基于时长的方法的性能。此外, 复合 LLM 应用的结构不确定性使传统 DAG 模型无法直接适用, 因为同一应用的不同作业可能具有不同拓扑, 且拓扑在运行时才能确定。这限制了拓扑感知调度器 [13] [22] 在新兴复合 LLM 应用中的使用。

为了解决上述挑战, 我们提出 LLMSched, 在服务提供方集群上高效调度在线到达的复合 LLM 应用。在 LLMSched 中, 我们首先设计新的基于 DAG 的模型, 解决描述复合 LLM 应用时的结构不确定性。该模型使用普通阶段、LLM 阶段、动态阶段以及阶段间依赖关系来表示一个复合 LLM 应用。在这一模型的支持下, 我们设计基于贝叶斯网络的性能分析器, 全面分析复合 LLM 应用的时长与结构, 并通过挖掘阶段间相关性, 识别降低不确定性的阶段。为了高效缓解不确定性, 我们进一步设计基于熵的机制, 量化调度这些阶段带来的不确定性降低。该机制使我们可以优先处理量化不确定性降低更大的阶段。由于主要目标是降低平均 JCT, 我们进一步使用 ϵ -greedy 算法, 将不确定性降低策略与高效优化 JCT 的调度方案结合, 提出不确定性感知调度器。为了大规模评估 LLMSched, 我们构建仿真器, 与多个最先进调度器比较, 并进一步实现真实测试平台进行实践测试。本文主要贡献如下:

- 我们对代表性复合 LLM 应用进行充分的数据分析,

¹在复合 LLM 应用中, 我们将具有特定用户输入的一个应用运行实例称为“作业”。

* 通讯作者。

识别出新兴复合 LLM 应用的时间与结构不确定性, 并揭示其调度挑战。

- 我们提出新的基于 DAG 的模型来描述复合 LLM 应用, 并提出基于贝叶斯网络的性能分析器, 分析其时长和结构, 识别降低不确定性的阶段。
- 我们设计基于熵的不确定性量化机制, 并通过审慎结合不确定性降低策略与高效优化 JCT 的方案, 设计不确定性感知调度器, 以降低平均 JCT。
- 大规模仿真和真实测试平台的实验结果表明, 相对于最先进调度方案, LLMSched 最高能够将平均 JCT 降低 79%。

本文其余部分安排如下: 第 II 节介绍复合 LLM 应用的背景、调度问题及现有调度策略; 第 III 节介绍工作负载特征与研究动机; 第 IV 节详细介绍 LLMSched 框架; 第 V 节给出实验结果; 第 VI 节讨论相关工作; 第 VII 节总结全文。

II. 背景

A. 复合 LLM 应用

我们全面回顾现有复合 LLM 应用, 深入分析其设计原则、执行工作流和实际实现。基于这一分析, 我们将其分为三类:

- **预定义应用** [8] [23] 具有显式定义的工作流, 阶段数量和依赖关系固定, 类似传统数据处理作业 [24] [25]。典型例子是 Graph-of-Thoughts [8] 中的序列排序。序列排序中, LLM 首先将原序列划分为多个部分; 对于每个部分, LLM 生成多个候选, 并结合用户定义函数, 对候选评分和选择; 随后 LLM 合并并处理所选候选, 得到最终排序结果。
- **链式应用** [9] [11] 逐步运行, 并遵循迭代操作模式。典型例子是代码生成 [9] [26]。给定用户查询后, 代码生成应用首先使用 LLM 生成若干代码测试用例, 再迭代执行代码生成 (LLM)、代码执行以及结果反思 (LLM)。迭代持续进行, 直到生成代码通过测试用例, 或者达到最大迭代次数。
- **规划应用** [18] [27] 要求 LLM 生成计划, 例如一组调用外部模块或 LLM 的阶段及其依赖关系, 再执行计划来完成用户查询。典型例子是任务自动化 [18]: LLM 分析用户任务, 选择深度学习模型等工具, 并识别所选工具之间的依赖关系。

B. 复合 LLM 应用的调度问题

本文关注在 GPU 机器集群上调度复合 LLM 应用, 以缩短响应时间。在一段时间内, 不同复合 LLM 应用的作业不断到达。事后观察, 每个作业可能具有多个阶段和阶段间依赖关系; 根据复合 LLM 应用的不同, 确切阶段及其依赖在开始时可能未知。根据阶段是否由 LLM 实现, 我们将阶段分别划分为 LLM 阶段和普通阶段, 对应任务分别称为 LLM 任务和普通任务。LLM 服务提供方部署普通执行器, 例如容器 [13] [24], 以及具有指定最大批大小的 LLM 执行器, 例如 LLM 引擎 [17], 来服务作业。每个普通执行器同一时刻只能执行一个普通任务; 每个 LLM 执行器可以同时执行多个 LLM 任务, 但不超过其最大批大小。我们的目标是调度到达的复合 LLM 应用作业, 最小化平均 JCT。

C. 现有调度策略及其不足

为了调度复合 LLM 应用, 考虑其固有的阶段间依赖, 可以使用不感知作业的方法 [24], 或者现有 DAG 调度方案 [13] [20] [22] [24]。先来先服务和公平调度 [24] 等不感知作业的方法, 依据到达时间或公平原则确定作业优先级, 不考虑作业信息。最短作业优先 [21] 和 Carbyne [20] 等基于时长的方法, 根据历史作业持续时间进行调度。Decima [13] 和 Argus [22] 等拓扑感知方法, 结合其 DAG 拓扑实现高效调度。

然而, 不感知作业的方法通常无法最优地降低平均 JCT。现有 DAG 调度方案通常依赖充分、准确的作业信息, 例如时长和 DAG 结构; 在调度复合 LLM 应用时, 这种假设并不充分 [13]。复合 LLM 应用固有的时长不确定性, 使时长估计不准确, 降低基于时长的方法的性能。此外, 结构不确定性也限制了拓扑感知方法的能力。现有调度方案的这些局限, 促使我们设计专用于复合 LLM 应用的新调度器。

III. 动机探索

A. 工作负载特征

本节分析复合 LLM 应用的运行时特征。我们选择上一节介绍的三种代表性复合 LLM 应用, 并在不同数据集上评估。对于预定义应用, 我们选择序列排序, 并使用包含 500 个随机生成序列的合成数据集, 序列长度从 16 到 64 不等。对于链式应用, 我们选择代码生成, 并在 MBPP 数据集 [28] 上运行; 该数据集包含 974 个编程任务, 通常用于评估 LLM 的代码生成性能。对于规划应用, 我们选择任务自动化, 并在 TaskBench [10] 上运行; 该基准通过数十种工具和数千个查询评估 LLM 的任务自动化能力。实验在配备一块 H800 GPU 的服务器上进行, 使用 Llama 2-7B 模型 [3], 批大小为 1。主要观察如下:

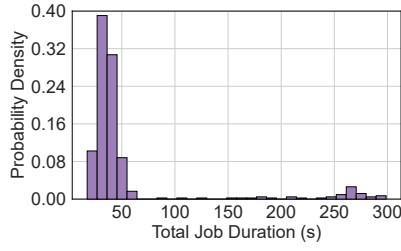
观察 1: 作业持续时间不确定。图 1a 展示了序列排序的作业持续时间分布。我们发现, 作业时长变化很大, 从 10 秒到接近 300 秒不等。这与作业时长通常稳定、确定的传统数据处理应用十分不同 [13] [20]。代码生成和任务自动化同样表现出较大的作业时长不确定性, 分别为 2 至 50 秒和 1 至 116 秒。由于篇幅限制, 此处不展示它们的可视化结果。我们的发现将此前对单个 LLM 请求时长的观察 [16] 扩展到复合 LLM 应用, 证实了其时间不确定性。

观察 2: 作业结构不确定。

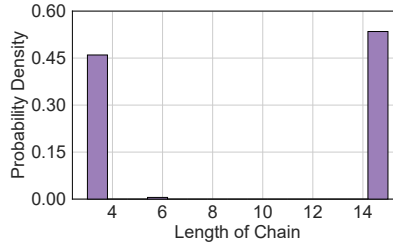
我们进一步考察代码生成和任务自动化的阶段数量, 并绘制其分布。代码生成根据代码执行结果, 遵循链式工作流, 链长度是动态的。如图 1b 所示, 链长度在 3 至 15 个阶段之间变化。同样, 我们发现, 任务自动化生成的阶段数量也具有动态性, 从 1 到 8 不等, 如图 1c 所示。这两个应用的阶段数量都不是预定义的, 而是在执行过程中逐步显现。这些结果揭示了复合 LLM 应用的结构不确定性。

B. 不确定性感知调度的动机

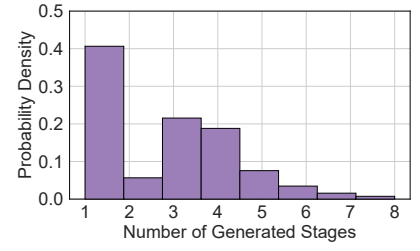
我们使用简单示例, 说明复合 LLM 应用的不确定性如何影响传统 DAG 调度方案的性能。考虑图 2 中的简单调度场景, 绿色块和米色块分别表示 LLM 阶段与普通阶段。图 2a 左侧展示任务自动化的通用工作流, 平均时长为 15 秒 (上方), 以及该应用的一个实际作业, 称为作业 1, 具有实际结构和 3 秒实际时长 (下方)。图 2a 右侧展示代码生成的通用工作流, 平均时长为 9 秒 (上方), 以及一个实



(a) 作业持续时间的不确定性。

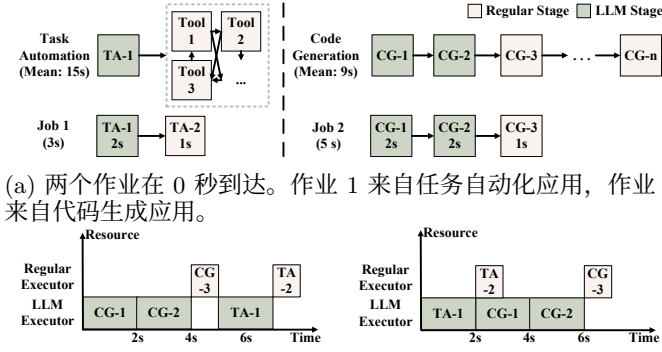


(b) 链长度的结构不确定性。



(c) 所生成阶段的结构不确定性。

图 1: 三种代表性复合 LLM 应用的运行时特征: (a) 序列排序; (b) 代码生成; (c) 任务自动化。



(a) 两个作业在 0 秒到达。作业 1 来自任务自动化应用, 作业 2 来自代码生成应用。
(b) 最短作业优先调度完成两个作业, 平均 JCT 为 6.5 秒。
(c) 不确定性感知调度完成两个作业, 平均 JCT 为 5 秒。

图 2: 考虑不确定性对复合 LLM 应用调度带来收益的示例。

际作业, 称为作业 2, 具有实际结构和 5 秒实际时长 (下方)。每个阶段标注了名称和时长, 例如 “TA-1 2s” 表示作业 1 中名为 “TA-1” 的阶段需要 2 秒完成, “CG-1 2s” 表示作业 2 中名为 “CG-1” 的阶段需要 2 秒完成。我们将 LLM 执行器数量、最大批大小和普通执行器数量均设为 1。两个作业均在 0 秒到达。

已知最短作业优先 (SJF) 方法能够高效最小化平均 JCT [20], 它按照估计时长调度作业。因此, 如图 2b 所示, 它根据各作业的期望时长, 先调度作业 2, 再调度作业 1。遗憾的是, 由于复合 LLM 应用的不确定性, 作业 2 的实际时长接近作业 1 的两倍, 与估计相反, 导致平均 JCT 高达 6.5 秒。

SJF 方法忽略复合 LLM 应用固有的不确定性, 因此取得次优性能。我们提出, 先调度能够降低不确定性的阶段, 就可以在这些阶段完成后获得有价值的信息, 再利用这些信息改善所有复合 LLM 应用作业的整体服务性能。如图 2c 所示, 我们识别出 TA-1 阶段与剩余未确定阶段高度相关, 因为作业 1 的其余部分完全取决于 TA-1 的 LLM 输出。完成 TA-1 后, 其时长和结构不确定性都被完全消除。因此, 我们先调度 TA-1, 发现作业 1 的时长远短于作业 2 的期望时长; 随后使用最短剩余时间优先 (SRTF) 调度剩余阶段。这一不确定性感知调度原则, 使平均 JCT 改善至 5 秒。

这一简单示例说明, 不确定性感知, 即识别并优先调度降低不确定性的阶段, 对于复合 LLM 应用调度具有重大意义。然而, 要将不确定性感知引入调度, 需要回答以下

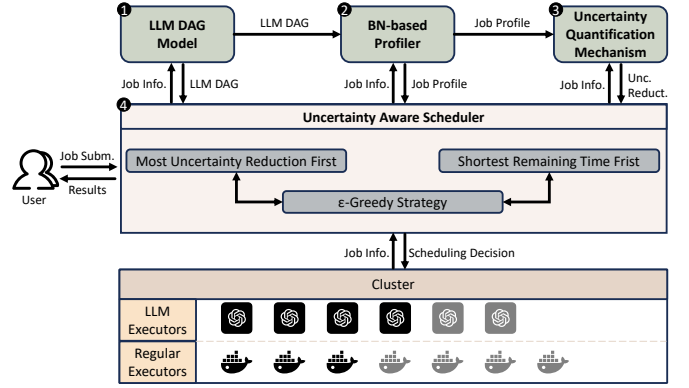


图 3: LLMSched 概览

问题: 1) 如何识别调度后能够降低不确定性的阶段? 2) 如果有多个降低不确定性的阶段, 如何决定先调度哪个? 3) 如何利用不确定性感知实现降低平均 JCT 的目标? 为了解决这三个问题, 接下来介绍 LLMSched 框架。

IV. LLMSCHED 框架

本节介绍 LLMSched, 一个面向复合 LLM 应用的高效调度框架。如图 3 所示, LLMSched 包含四个主要组成部分: 第一, 新的基于 DAG 的模型, 用于描述复合 LLM 应用; 第二, 基于贝叶斯网络的性能分析器, 用于分析复合 LLM 应用并识别降低不确定性的阶段; 第三, 基于熵的不确定性量化机制; 第四, 用于高效调度的不确定性感知调度器。

A. 复合 LLM 应用的 DAG 模型

本节介绍用于描述复合 LLM 应用的 DAG 模型, 作为解决第 III 节三个问题的基础。由于复合 LLM 应用的结构不确定性, 以及 LLM 推理任务的独特特征, 传统 DAG 格式不再适用。因此, 我们设计新的基于 DAG 的模型, 解决描述这些应用时的结构不确定性。该模型使用三类阶段刻画现有复合 LLM 应用, 将各类阶段作为节点, 将阶段间依赖作为边。三类阶段如下:

- **普通阶段:** 一个普通阶段包含一个或多个普通任务, 即非 LLM 任务, 与传统 DAG 模型中的阶段定义相同 [24]。普通任务在容器等普通执行器上运行 [27]。
- **LLM 阶段:** 一个 LLM 阶段包含一个或多个 LLM 推理任务。LLM 任务在 LLM 执行器上运行, 每个执行器运行一个 LLM 实例 [17]。

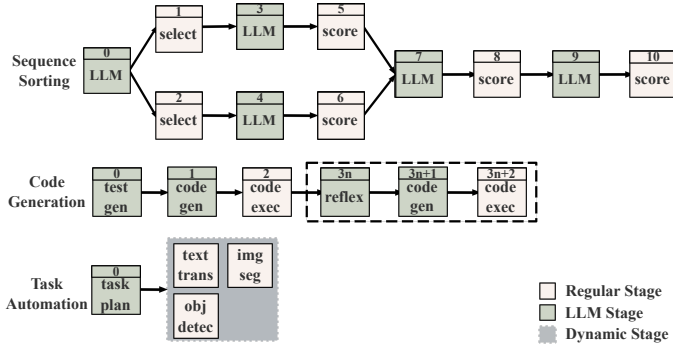


图 4: 序列排序、代码生成和任务自动化的 LLM DAG 表示。

- **动态阶段:** 一个动态阶段充当 LLM 生成阶段及其依赖关系的占位符；这些阶段由前面的 LLM 阶段从候选集合中构造。候选集合包含供 LLM 调用的其他普通阶段或 LLM 阶段。

如第 III 节所述，链式应用和规划应用存在结构不确定性。对于现有链式应用，我们注意到，它们通常按照固定模式迭代，并规定最大迭代次数。因此，可以根据该模式和最大迭代次数填充链，从而在描述层面解决结构不确定性。对于规划应用，动态阶段的设计直接抽象了其不确定性。

示例说明: 图 4 展示如何使用该模型，将复合 LLM 应用描述为 DAG，并给出第 III 节三种应用的表示。每个阶段的数字，表示按拓扑顺序排序得到的阶段 ID。第一个应用是序列排序，具有预定义的显式依赖，因此使用原论文 [8] 定义的结构。第二个例子是代码生成；由于篇幅限制，只展示最大迭代次数为 1 的表示。一轮迭代中的阶段用黑色虚线矩形圈出。对于任务自动化，我们使用一个 LLM 阶段进行任务规划，再使用一个动态阶段表示所生成计划；它的阶段候选集合包含文本翻译、图像分割和目标检测三个普通阶段。

B. 基于贝叶斯网络的性能分析器

本节回答第一个问题：如何识别调度后能够降低不确定性的阶段？第 III 节的动机示例展示了：调度与动态阶段相关的 LLM 阶段能够降低不确定性。受此启发，我们尝试利用阶段间相关性识别降低不确定性的阶段。在规划应用中，动态阶段与其前驱 LLM 阶段之间的相关性显而易见，因为动态阶段中的阶段及依赖，由 LLM 阶段的输出决定。因此，我们进一步研究预定义和链式应用中普通阶段与 LLM 阶段内部及相互之间的相关性。为此，我们绘制两种代表性应用各阶段时长的热图：预定义的序列排序，以及链式的代码生成²。热图使用 Pearson 系数，接近 1 或 -1 的值分别表示正或负线性相关。如图 5 所示，一些阶段的时长表现出较强相关性。例如，序列排序的阶段 0 与 3 的系数为 0.7，代码生成的阶段 3 与 6 的系数接近 0.9。

不同阶段时长之间的强相关性，反映了复合 LLM 应用内在的设计逻辑。在序列排序中，LLM 在阶段 0 将原始序列分为相等的两部分，并在阶段 3 和 4 处理这些部分，

²由于代码生成的不同作业具有不同迭代次数，我们在运行时，将未执行阶段的时长视为 0 秒。

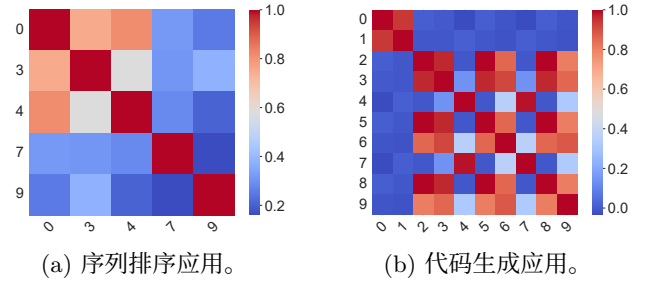


图 5: 两种复合 LLM 应用的阶段时长热图。坐标轴记录按照图 4 中 DAG 的拓扑顺序排序得到的阶段 ID。

生成排序结果。由于生成 Token 数量存在比例关系，这些阶段具有较高相关系数。代码生成的相关性来自两个因素：首先，图 4 中的每个 *code gen* 阶段，都根据执行结果修改先前生成的代码，因此代码和时长相似；其次，迭代性质保证标准迭代中的各个阶段顺序执行，例如一个 *reflex* 阶段意味着随后会有 *code gen* 和 *code exec* 阶段。

传统调度方案 [13] [20] 针对单个作业或单个阶段分析时长，因此错过了利用阶段间相关性降低不确定性的机会。为了全面分析复合 LLM 应用，捕获相关性并识别降低不确定性的阶段，我们采用贝叶斯网络 (BN) [29]。BN 是一种概率图模型，使用 DAG 表示一组变量及其条件依赖。将复合 LLM 应用中每个阶段的时长分布离散化后，可以训练 BN，得到阶段时长的条件概率分布 (CPD) 及其相关性。CPD 指定给定网络中父变量时某个变量的概率分布，从而能够将每个阶段的时长建模为依赖前驱阶段时长的变量。BN 中的边进一步表示这些依赖，揭示阶段间相关性，并识别降低不确定性的阶段。具体而言，给定 BN 中两个阶段 u 和 v ，若二者之间存在有向路径，则认为 u 与 v 相关：

$$\text{correlated}(u, v) = \begin{cases} 1 & \text{if } u \xrightarrow{*} v, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

我们将所有与一个或多个其他阶段相关的阶段，视为降低不确定性的阶段。

利用 BN，我们还能动态更新作业时长与结构的分析结果，使估计更加准确。具体做法是使用已完成阶段的时长以及获得的 CPD，推断未完成阶段的后验分布。例如，若较早阶段比预期耗时更长，BN 就能够据此调整后序阶段的估计时长。BN 的推断能力支持实时调整和更加精确的调度决策，从而降低不确定性、提高效率。

感知批处理的时长校准: 如前文所述，LLM 任务的时长会受到运行时批处理请求数量的影响，加剧时长不确定性，并可能导致时长估计不准确。因此，我们提出感知批处理的时长校准，根据当前批处理请求调整 LLM stage 的时长估计，辅助 BN 在运行时估计作业时长。我们仿真具有不同批大小的工作负载，并记录解码一个 Token 的平均延迟。对于时长为 d_r 、在批大小 b_r 下执行的 LLM 任务，我们将其估计时长 d_t 在目标批大小 b_t 下校准如下：

$$d_t = d_r \times \frac{l(b_t)}{l(b_r)}, \quad (2)$$

其中， $l(b_t)$ 和 $l(b_r)$ 是在相应批大小下记录的平均解码延迟。

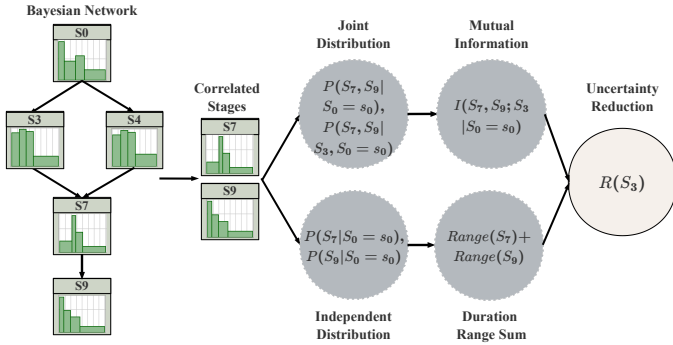


图 6: 计算调度序列排序应用中 S_3 阶段所带来的不确定性降低的示例。

C. 基于熵的不确定性量化

本节回答第二个问题：若有多个降低不确定性的阶段，如何决定先调度哪个？BN 得到的 CPD 启发我们使用 Shannon 熵 [30]；这是一种对分布敏感、用于度量随机变量不确定性的指标，可以量化调度某个阶段带来的不确定性降低。给定具有 n 个可能取值的离散随机变量 X ，其熵 $H(X)$ 定义为：

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i), \quad (3)$$

其中， $p(x_i)$ 是 X 取值为 x_i 的概率。

为了利用 Shannon 熵量化不确定性，我们首先设计随机变量，分别刻画 DAG 模型中的三类阶段：

- **普通阶段的不确定性** 在于该阶段是否执行，因为普通任务的时长通常是确定的 [13] [19]。可以直接使用二元随机变量表示其执行情况。
- **LLM 阶段的不确定性** 同时存在于是否执行以及持续时间两个方面。将时长分布离散化为 k 个区间，并将不执行视为时长 0 秒，就可以构造具有 $k+1$ 个不同取值的随机变量来表示 LLM 阶段。
- **动态阶段的不确定性** 在于其内部阶段及依赖关系，可以通过候选集中阶段和边的选择来刻画。我们将动态阶段的不确定性，定义为节点，即阶段的熵与边，即依赖关系的熵之和 [31]，表示如下：

$$H(X) = \sum_{c \in \mathcal{C}} H(I_c) + \sum_{e \in \mathcal{E}} H(I_e), \quad (4)$$

其中， $\mathcal{C}$ 是阶段候选集合， $\mathcal{E}$ 是候选集合 $\mathcal{C}$ 中阶段之间可能的边集合， I_c 和 I_e 是二元变量，分别表示阶段 c 和边 e 是否存在。

接下来，我们说明如何量化调度阶段 X 带来的不确定性降低。设 Y 为与 X 相关的尚未调度阶段；首先引入互信息 $I(Y; X)$ ，度量获知 X 能在多大程度上降低关于 Y 的熵：

$$I(Y; X) = - \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)} \quad (5)$$

其中， x 和 y 分别是 X 和 Y 可能的离散化时长取值，例如时长区间。为了纳入所有相关阶段的时长变化，我们进

算法 1 不确定性感知调度

输入 未完成 LLM 作业列表 J ，探索概率 $\epsilon \in [0, 1]$ ，任务采样比例 $r \in [0, 1]$ 。

输出 可调度 LLM 任务和普通任务的调度优先顺序。

```

1:  $J_t = \text{sort}(J, \text{by} = \text{job.est\_rd}())$ 
2: for all  $\text{job}$  in  $J_t$  do
3:    $S_t.append(\text{job.ready\_stages}())$ 
4: end for
5: 找出  $J$  的互不重叠集合  $J_{no}$ 。
6: for all  $j_{no}$  in  $J_{no}$  do
7:   取得  $j_{no}$  中每个作业的可调度阶段，记为  $s_{no}$ 。
8:    $s'_{no} = \text{sort}(s_{no}, \text{by} = \text{stage.uncert\_reduct}())$ 
9:    $S_u.extend(s'_{no})$ 
10: end for
11: while  $S_t.has\_stage()$  and  $S_u.has\_stage()$  do
12:    $s_t = S_t.pop()$ ,  $s_u = S_u.pop()$ 
13:    $p = \text{random}(0, 1)$ 
14:   if  $p \leq \epsilon$  then
15:      $t = s_u.sample\_tasks(r)$ 
16:   else
17:      $t = s_t.all\_tasks()$ 
18:   end if
19:   若  $t$  为 LLM 任务，则将  $t$  加入  $T_l$ ，否则加入  $T_r$ 。
20: end while
21: 将剩余任务加入  $T_l$  或  $T_r$ 。
22: 返回  $T_r$ 、 $T_l$ 。

```

一步考虑所有相关阶段分布取值范围之和，细化熵减度量，计算最终的不确定性降低：

$$R(X) = I(Y_1, \dots, Y_M; X | \mathbf{E}) \times \sum_{m=1}^M \text{Range}(Y_m), \quad (6)$$

其中， $Y_1, \dots, Y_M$ 是与 X 相关的尚未调度阶段， $\mathbf{E}$ 是所有已完成阶段时长的证据集合。对于动态阶段，其不确定性会在前驱 LLM 阶段完成时消除，因此我们将使用式(4)度量的动态阶段不确定性，乘以其时长分布的取值范围，再将结果加到前驱 LLM 阶段的不确定性降低量中。

示例说明：图 6 使用序列排序应用说明如何计算调度一个阶段带来的不确定性降低。图左侧展示学习得到的序列排序 BN。假设已完成 S_0 ，得到其执行时长 s_0 ，现在要计算调度 S_3 带来的不确定性降低。首先根据式(1)，收集与 S_3 相关的全部阶段，即 S_7 和 S_9 。根据式(5)和(6)，从 BN 得到两个联合时长分布，即 $P(S_7, S_9 | S_0 = s_0)$ 和 $P(S_7, S_9 | S_3, S_0 = s_0)$ ，并用它们计算阶段 S_7 、 S_9 与 S_3 之间的互信息。随后获得每个相关阶段各自的时长分布，即 $P(S_7 | S_0 = s_0)$ 和 $P(S_9 | S_0 = s_0)$ ，计算分布取值范围之和。最后将互信息与范围之和相乘，得到调度 S_3 的最终不确定性降低量。

D. 不确定性感知调度器

本节回答最后一个问题：如何利用不确定性感知，达到降低平均 JCT 的目标？我们的思路是，将降低作业不确定性的过程视为探索，将利用已获得作业信息来识别并调度小作业的过程视为利用。由此，我们提出不确定性感知调



图 7: 四种工作负载的仿真结果 ($\lambda = 0.9$)。规划工作负载中省略 Decima, 因为其平均 JCT 过高, 超过 100 秒。

度器, 以探索与利用相结合的方式, 将不确定性降低策略与高效优化 JCT 的最短剩余时间优先 (SRTF) 方案结合。

算法 1 给出细节: 输入未完成作业列表 J 、探索概率 ϵ 和任务采样比例 r , 输出两个调度优先列表, 普通任务列表 T_r 和 LLM 任务列表 T_l 。首先, 使用高效优化 JCT 的方案, 根据各阶段所属作业的估计剩余时长排序, 创建列表 S_t (第 1 至 4 行)。该估计通过 CPD 和式(2)中的校准机制, 计算作业时长分布的均值得到。

随后, 使用不确定性降低策略, 根据阶段的量化不确定性降低量, 创建另一个阶段列表 S_u (第 5 至 10 行)。由于复合 LLM 应用具有不确定性, 一个作业的时长分布可能与其他作业重叠, 因此按照估计时长排序可能出现错误。如果每个作业时长分布的区间都与其他作业不重叠, 就能保证排序准确率为 100%。为此, 我们首先根据时长分布区间, 将全部作业划分为多个互不重叠集合 (第 5 行)。各集合按下界排序; 在每个集合内部, 根据各作业阶段的不确定性降低量排序, 再将排序结果依次加入 S_u (第 6 至 10 行)。

现在, 我们有两个不同的阶段排序列表: 一个根据估计作业时长排序, 另一个根据不确定性降低量排序。二者体现两个调度策略的双重目标: 最小化平均 JCT, 以及降低作业不确定性。为解决可能的调度冲突, 我们采用 ϵ -greedy 算法 [32], 以探索与利用方式组合两个策略 (第 11 至 20 行)。具体而言, 每次迭代以概率 ϵ 调度 S_u 中的阶段, 以概率 $1 - \epsilon$ 调度 S_t 中的阶段 (第 13 至 18 行)。这一设计平衡了利用当前信息降低平均 JCT, 以及探索新信息降低不确定性。然而, 若选中探索的阶段包含过多任务, 可能显著推迟小作业执行, 损害平均 JCT。因此, 我们仅采样比例为 r 的任务来调度, 而非调度全部任务 (第 15 行); 这样, 只运行一小部分任务, 就能估计整个阶段的时长。最后, 将剩余任务附加到 T_r 和 T_l 末尾, 作为最终调度优先列表返回 (第 21 至 22 行)。对于多个 LLM 执行器, 我们采用简单负载均衡方法, 将下一个要调度的 LLM 任务分配给运行中任务数量最少的实例; 这一方法能适应多种类型和数量的复合 LLM 应用。

复杂度分析: 假设执行算法 1 时, 系统中有 N 个阶段。根据估计剩余时长排序作业 (第 1 行), 需要 $O(N \log_2 N)$; 收集全部可调度阶段 (第 2 至 4 行), 需要 $O(N)$; 找出作业互不重叠集合 (第 5 行), 需要 $O(N)$; 按照不确定性降低量排序阶段 (第 6 至 10 行), 需要 $O(N \log_2 N)$; 对全部阶段进行任务采样 (第 11 至 20 行), 需要 $O(N)$ 。因此, 算法 1 的时间复杂度为 $O(N \log_2 N)$, 能够扩展至大量、不同类型的复合 LLM 应用。

上述分析未计入贝叶斯推断开销; 该推断用于计算作业

剩余时长 (第 1 行) 和各阶段不确定性降低量 (第 8 行)。这一省略有两个理由: 首先, 为不同阶段离散取值的所有可能组合维护查找表, 就能将贝叶斯推断降至 $O(1)$; 其次, 为优化推断成本和提高效率, 现有复合 LLM 应用通常设计为少于 10 个 LLM 阶段 [10] [12] [27]。这些阶段构成 BN 中的大多数节点, 且并非所有阶段都纳入 BN, 因此在如此小的网络上, 贝叶斯推断实际上可视为常数时间。测试平台实验 (第 V-B 节) 记录的调度延迟证实了这一低开销。

V. 评估

实现: 我们使用 Python 3.9, 通过约 3 千行代码实现 LLMSched。我们在相应数据集上记录不同复合 LLM 应用的运行时长, 并使用 PyAgrum [33] 训练贝叶斯网络; 任务自动化除外, 因为它只包含一个 LLM 阶段和一个动态阶段。根据频率, 每个阶段的时长分布最多离散化为 6 个区间。对于 LLM, 我们使用 Llama2-7B 模型 [3], 以 vLLM [17] 作为服务引擎。

工作负载生成: 我们从第 II 节三类应用中, 选择 6 种代表性复合 LLM 应用: (1) 预定义应用, 包括使用合成数据集的序列排序 [8], 以及使用原论文数据集的文档合并 [8]; (2) 链式应用, 包括代码生成 [9], 使用 MBPP [28] 数据集, 以及网页搜索 [11], 使用 HotpotQA [34] 数据集; (3) 规划应用, 包括任务自动化 [10], 使用 TaskBench [10] 数据集, 以及 LLMCompiler [27], 使用 HotpotQA [34] 数据集。

使用这六种复合 LLM 应用, 我们构造四类工作负载: (1) **混合工作负载**, 六种应用的作业数量均匀分布; (2) **预定义工作负载**, 50% 为序列排序, 50% 为文档合并; (3) **链式工作负载**, 50% 为代码生成, 50% 为网页搜索; (4) **规划工作负载**, 50% 为任务自动化, 50% 为 LLMCompiler。

对于每类工作负载, 我们从相应数据集随机采样查询, 使作业到达服从到达率为 λ 的泊松分布 [13]。

测试平台: 真实实验在配备 NVIDIA H800-100G GPU、AMD EPYC 9654 处理器及 120 GB 内存的服务器上进行。

仿真器: 我们还构建仿真器进行大规模评估。实验观察表明, 批大小主要影响解码延迟, 而批处理中 Token 数量等其他因素影响很小。因此, 我们分析解码延迟与批大小之间的关系, 并依据分析结果, 在并发运行请求数量每次变化时, 动态调整每个正在运行 LLM 任务的剩余时长。

基线: 我们将 LLMSched 的性能与以下六个基线比较:

- 先来先服务 (FCFS): 按到达顺序调度作业, 是 Spark 的默认方案。
- 公平调度 (Fair): 向每个运行中作业分配相同数量的资源, Spark 支持该策略。

- 最短作业优先 (SJF): 优先处理估计时长最短的作业。
- Argus [22]: 根据阶段深度、子节点数量以及任务数量, 对阶段排序。
- Decima [13]: 使用强化学习 (RL) 学习 DAG 作业的调度策略。为了公平比较, RL 智能体在上述四类工作负载上训练。
- Carbyne [20]: 通过收集并重新分配剩余资源, 以利他方式调度作业。

对于基线所需的先验信息, 我们在各应用的数据集上, 计算平均时长和资源需求, 并从所提出的 DAG 模型提取 DAG 结构。

参数设置: 默认参数为 300 个作业, 作业到达率为 0.9, 与 [35] 一致。我们为四类工作负载手动配置 LLM 执行器和普通执行器资源, 使平均集群负载达到适中的 85%, 与 [13] 一致。

A. 仿真结果

我们在图 7 中给出不同作业数量的仿真结果。就平均 JCT 而言, LLMSched 优于全部其他基线。对于四类工作负载, LLMSched 分别可将平均 JCT 降低 36~79%、14~46%、36~67% 和 24~52%。此外, 作业数量增加时, 其优势更加明显。例如, 相对于 Decima, 对于具有 100、200、300 和 400 个作业的混合工作负载, LLMSched 分别降低平均 JCT 38%、65%、73% 和 75%, 展示了其可扩展性。

接下来详细分析各方法在四类工作负载中的性能。FCFS 等不感知作业的方法, 以及 Fair、Carbyne 等基于公平性的方法, 在四类工作负载中表现均较差, 因为它们未优化平均 JCT, 而这是复合 LLM 应用最关键的因素。对于混合工作负载, 不同复合 LLM 应用的作业时长差异显著; SJF 通过优先处理较短作业, 优于其他基线。然而, 它仍落后于 LLMSched, 因为没有考虑复合 LLM 应用的不确定性, 可能导致作业优先级错误。对于预定义工作负载, 由于具有固定 DAG 结构, Argus 凭借拓扑感知能够高效调度。然而, 同一应用的作业共享相同拓扑, 因此 Argus 实际上是在应用层面调度; 由于时长不确定性, 这对于平均 JCT 而言并非最优。相比之下, LLMSched 使用 BN 更新剩余时长估计, 确定同一应用内部各作业的优先级, 从而优于 Argus。

对于同时具有时长和结构不确定性的链式工作负载, LLMSched 凭借处理复合 LLM 应用固有不确定性的优势, 大幅优于其他基线。对于规划工作负载, 作业具有较高阶段并行度, 例如 LLMCompiler [27] 中的多个并行工具调用, 但任务并行度较低, 例如每个阶段只包含一个任务; Decima 表现最差, 因为它每次只调度一个阶段的任务, 使资源利用不足。LLMSched 仍表现最好, 因为其探索与利用策略先揭示作业的动态阶段以缓解不确定性, 再使用得到的信息更高效地调度。

B. 测试平台结果

图 8 展示测试平台中, 各方法针对四类工作负载的平均 JCT。相对于其他六个基线, LLMSched 分别将四类工作负载的平均 JCT 降低 45~66%、26~46%、35~45% 和 38~51%。测试平台中的比较结果, 与仿真环境几乎一致, 验证了仿真器的准确性。

表 I: 各方法的平均调度开销, 单位为毫秒。

	混合	预定义	链式	规划
FCFS	0.25	0.47	0.06	0.1
SJF	0.41	0.66	0.17	0.18
FAIR	0.48	0.7	0.08	0.12
Argus	0.32	0.53	0.07	0.1
Decima	17.7	28.79	12.68	10.17
Carbyne	4.39	8.23	0.6	0.62
LLMSched	0.96	2.32	0.7	0.16

我们还在表 I 中给出测试平台实验中各方法的平均调度开销, 通过将每种方法的总开销除以调用次数得到。LLMSched 的开销也包含 BN 推断和熵计算。我们方法的平均开销略高于 FCFS、SJF 和 Fair 等简单启发式方法, 但远低于 Decima、Carbyne 两种复杂方法。此外, LLMSched 在所有工作负载中的平均开销均低于 3 毫秒, 表明它能够高效实时调度, 而不会影响平均 JCT。

C. 消融研究

LLMSched 框架包含两个关键组成部分: 一个是更新后继阶段分布估计的贝叶斯网络, 另一个是不确定性感知调度策略。为了分析两者的有效性, 我们进行消融研究, 构造两个额外方法: *LLMSched w/o BN* 和 *LLMSched w/o uncertainty*。前者遵循算法 1 中的相同调度方案, 但使用历史作业平均时长进行估计; 后者使用贝叶斯网络更新作业时长后验分布, 但只执行 SRTF 策略。

图 10 展示四类工作负载的消融结果。为了比较, 将两个方法的平均 JCT 以 LLMSched 的结果归一化。对于 *LLMSched w/o BN*, 四类工作负载的平均 JCT 分别比 LLMSched 高 18%、17%、20% 和 5%。这表明 BN 发挥重要作用, 显著提高作业时长估计的准确性; 利用 BN 的阶段间相关性, 可以动态更新尚未调度阶段的作业时长估计, 并更加准确地预测。对于 *LLMSched w/o uncertainty*, 四类工作负载的平均 JCT 分别比 LLMSched 高 21%、12%、15% 和 13%。这表明不确定性感知策略对于有效指导探索过程至关重要。在混合工作负载中, 各阶段的不确定性降低量差异显著, 其重要性尤为突出; 在这一工作负载下, *LLMSched w/o BN* 的表现优于 *LLMSched w/o uncertainty*。

D. 敏感性分析

本节分析不同探索概率 ϵ 、任务采样比例 r 和作业到达率 λ 下 LLMSched 的性能。图 9a 展示探索概率与归一化平均 JCT 的关系。随着概率增加, 平均 JCT 先下降, 再逐渐上升, 说明探索与利用之间存在权衡, 需要取得平衡。

图 9b 展示任务采样比例与归一化平均 JCT 的关系。比例过小时, 作业时长估计误差可能较大, 因而增加平均 JCT; 比例增大时, 任务采样获得更多资源, 小作业执行被推迟, 同样会导致平均 JCT 较高。因此, 适当较小的采样比例更为合适。

图 9c 展示四类工作负载在作业到达率为 0.6、0.9 和 1.2 时的归一化 JCT, 对应模拟轻负载、中负载和重负载集群环境。总体而言, 作业到达越频繁, 平均 JCT 越大。对于链式和规划工作负载, LLMSched 的平均 JCT 变化更显著; 对于混合和预定义工作负载, 变化更平缓。

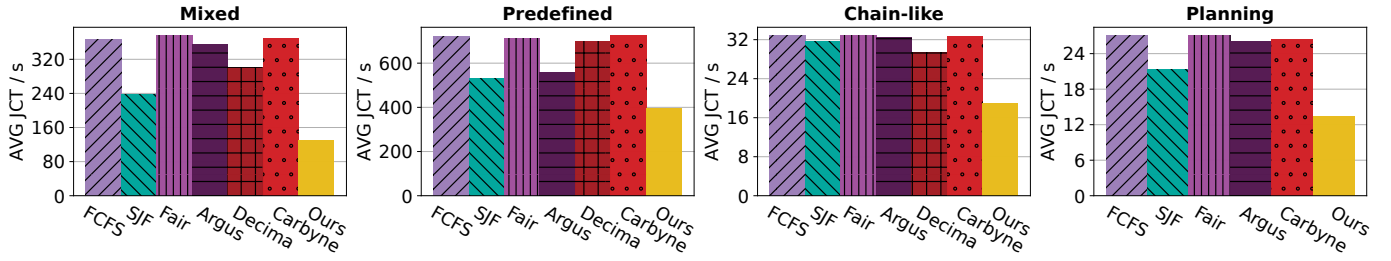


图 8: 不同方法针对四类工作负载的测试平台结果 ($\lambda = 0.9$, 300 个作业)。LLMSched 标记为 *Ours*。规划工作负载中省略 Decima, 因为其平均 JCT 过高, 超过 100 秒。

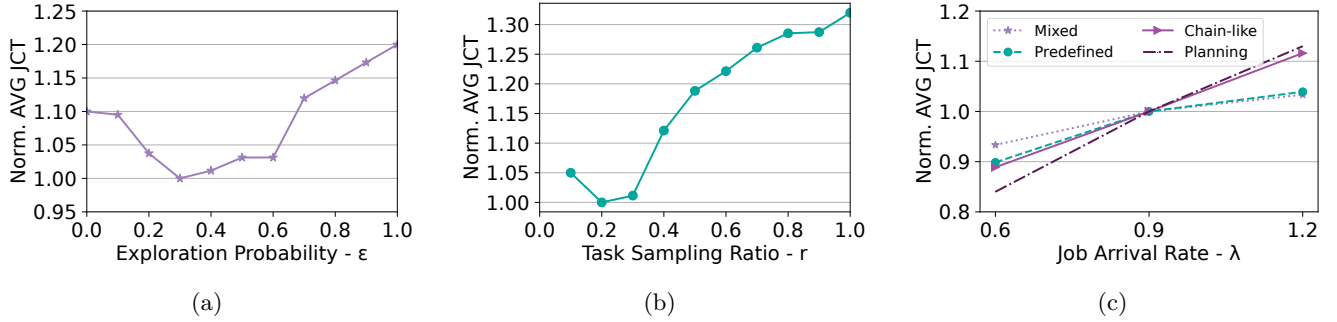


图 9: LLMSched 在不同探索概率、任务采样比例和作业到达率下的敏感性分析。

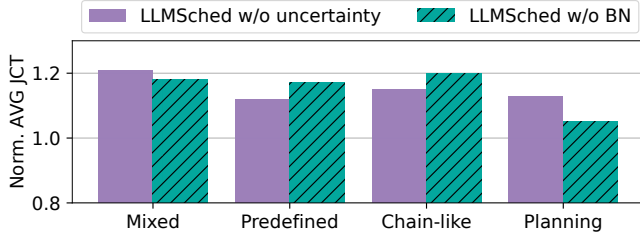


图 10: LLMSched 在四类工作负载上的消融研究。

VI. 相关工作

A. DAG 作业调度

针对具有有向无环图 (DAG) 结构的数据处理作业, 降低平均作业完成时间 (JCT) 或整体完工时间的调度问题, 已经得到广泛研究。大多数现有工作 [19] [13] [14], 利用重复作业充分的先验知识, 即 DAG 结构和作业时长进行调度。Graphene [19] 优先处理运行时间长或难以放置的棘手作业; Decima [13] 使用图神经网络捕获 DAG 拓扑, 并通过强化学习学习调度策略; Branch scheduling [14] 将 DAG 阶段组织为分支, 降低各作业内的数据传输开销。这些方法依赖准确的作业时长与 DAG 结构信息, 而复合 LLM 应用往往无法提供这些信息。我们的框架能够高效处理复合 LLM 应用中的不确定性, 并成功改善其调度性能。

B. LLM 服务

为了加快 LLM 推理速度、提高吞吐量, 已有多项工作 [15] [16] [36] [17]。Orca [15] 在迭代层面而非请求层面调度 LLM 请求, 并进行选择性批处理。FastServe [16] 使用抢占式调度来最小化响应时间, 采用新的跳级加入多

级反馈队列调度器。FlashAttention [36] 通过分块减少内存读写次数, 提高 LLM 推理速度。vLLM [17] 采用新的 PagedAttention 管理 LLM 中 KV cache 的内存使用, 提高吞吐量。这些工作只关注单请求服务, 而我们提出首个专门面向新兴复合 LLM 应用的不确定性感知调度框架。

VII. 结论

本文提出 LLMSched, 一个面向复合 LLM 应用的不确定性感知调度框架。我们对现有代表性复合 LLM 应用进行数据分析, 揭示其时长和结构不确定性, 以及调度挑战。随后, 我们设计新的 DAG 模型和基于贝叶斯网络的性能分析器, 捕获这些不确定性, 并引入基于熵的机制进行量化。接着, 提出不确定性感知调度方案, 以降低平均 JCT。仿真环境和测试平台的实验结果均证明, 所提出方法优于最先进调度算法。

VIII. 致谢

本研究获得中国国家重点研发计划资助, 项目编号为 2024YFC3017100。

参考文献

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, Mar. 2023.
- [2] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth *et al.*, “Gemini: a family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, Dec. 2023.
- [3] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, Feb. 2023.

- [4] (2023) Vicuna: An open-source chatbot impressing gpt-4 with 90% chatgpt quality. [Online]. Available: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [5] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, Sep. 2023.
- [6] (2024) 107 up-to-date chatgpt statistics & user numbers [april 2024]. [Online]. Available: <https://nerdynav.com/chatgpt-statistics/>
- [7] Y. Zhu, B. Zhu, C. Chen, and X. Fan, “Towards efficient compound large language model system serving in the wild,” in *Proc. of the Int. Symp. on Qual. of Serv. (IWQoS)*, Jun. 2024, pp. 1–2.
- [8] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, “Graph of thoughts: Solving elaborate problems with large language models,” in *Proc. AAAI Conf. on Artif. Intel. (AAAI)*, Mar. 2024, pp. 17 682–17 690.
- [9] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Annu. Conf. on Neural Inf. Process. Sys. (NeurIPS)*, Dec. 2024.
- [10] Y. Shen, K. Song, X. Tan, W. Zhang, K. Ren, S. Yuan, W. Lu, D. Li, and Y. Zhuang, “Taskbench: Benchmarking large language models for task automation,” *arXiv preprint arXiv:2311.18760*, Nov. 2023.
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Int. Conf. on Learn. Represent. (ICLR)*, May 2023.
- [12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, “Autogen: Enabling next-gen llm applications via multi-agent conversation framework,” *arXiv preprint arXiv:2308.08155*, Aug. 2023.
- [13] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proc. of the ACM Spec. Int. Group on Data Commun. (ACM SIGCOMM)*, Aug. 2019, pp. 270–288.
- [14] Z. Hu, D. Li, Y. Zhang, D. Guo, and Z. Li, “Branch scheduling: Dag-aware scheduling for speeding up data-parallel jobs,” in *Proceedings of the international symposium on quality of service*, Aug. 2019, pp. 1–10.
- [15] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for transformer-based generative models,” in *USENIX Symp. Operating Syst. Des. Implementation (OSDI)*, Oct. 2022, pp. 521–538.
- [16] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, “Fast distributed inference serving for large language models,” *arXiv preprint arXiv:2305.05920*, May 2023.
- [17] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proc. 29th Symp. Operating Syst. Princ. (SOSP)*, Oct. 2023, pp. 611–626.
- [18] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face,” in *Annu. Conf. on Neural Inf. Process. Sys. (NeurIPS)*, Dec. 2024.
- [19] R. Grandl, S. Kandula, S. Rao, A. Akella, and J. Kulkarni, “Graphene: Packing and dependency-aware scheduling for data-parallel clusters,” in *USENIX Symp. Operating Syst. Des. Implementation (OSDI)*, Nov. 2016, pp. 81–97.
- [20] R. Grandl, M. Chowdhury, A. Akella, and G. Ananthanarayanan, “Altruistic scheduling in multi-resource clusters,” in *USENIX Symp. Operating Syst. Des. Implementation (OSDI)*, Nov. 2016, pp. 65–80.
- [21] K. Agrawal, J. Li, K. Lu, and B. Moseley, “Scheduling parallel dag jobs online to minimize average flow time,” in *Proc. 27th Annu. ACM-SIAM Symp. Discrete Algorithms (ACM-SIAM SDA)*, Oct. 2016, pp. 176–189.
- [22] S. Wu, H. Chen, Y. Wang, and H. Jin, “Argus: Efficient job scheduling in rdma-assisted big data processing,” in *IEEE Int. Paralle. and Distrib. Process. Symp. (IPDPS)*, May 2021, pp. 827–836.
- [23] (2024) Langchain - build context-aware reasoning applications. [Online]. Available: <https://python.langchain.com/>
- [24] (2024) Apache spark. [Online]. Available: <https://spark.apache.org/>
- [25] (2024) Apache hadoop. [Online]. Available: <https://hadoop.apache.org/>
- [26] M. M. Rahman, A. Kundu, and E. Bertino, “Benchmarking of code generative llms,” in *2024 IEEE 44th Int. Conf. on Dist. Comp. Syst. (ICDCS)*. IEEE, Jul. 2024, pp. 1448–1449.
- [27] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. Mahoney, K. Keutzer, and A. Gholami, “An llm compiler for parallel function calling,” *arXiv*, Oct. 2023.
- [28] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, “Program synthesis with large language models,” *arXiv preprint arXiv:2108.07732*, Aug. 2021.
- [29] N. Friedman, D. Geiger, and M. Goldszmidt, “Bayesian network classifiers,” *Machine learning*, vol. 29, no. 2-3, pp. 131–163, Nov. 1997.
- [30] C. E. Shannon, “A mathematical theory of communication,” *The Bell system technical journal*, vol. 27, pp. 379–423, Jul. 1948.
- [31] A. K. Wong and M. You, “Entropy and distance of random graphs with application to structural pattern recognition,” *IEEE Trans. on Pattern Anal. and Mach. Intell. (TPAMI)*, no. 5, pp. 599–609, Sep. 1985.
- [32] J.-Y. Audibert and S. Bubeck, “Best arm identification in multi-armed bandits,” in *COLT-23th Conference on learning theory-2010*, 2010, pp. 13–p.
- [33] G. Ducamp, C. Gonzales, and P.-H. Wuillemin, “agrum/pyagrum: a toolbox to build models and algorithms for probabilistic graphical models in python,” in *Int. Conf. on Probabil. Graph. Models (ICPGM)*, Sep. 2020, pp. 609–612.
- [34] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, “Hotpotqa: A dataset for diverse, explainable multi-hop question answering,” in *Proc. of the Conf. on Empir. Methods in Nat. Lang. Process. (EMNLP)*, Oct. 2018, pp. 2369–2380.
- [35] X. Miao, G. Oliaro, X. Cheng, M. Wu, C. Unger, and Z. Jia, “Flexllm: A system for co-serving large language model inference and parameter-efficient finetuning,” *arXiv preprint arXiv:2402.18789*, Feb. 2024.
- [36] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Annu. Conf. on Neural Inf. Process. Sys. (NeurIPS)*, vol. 35, pp. 16 344–16 359, Dec. 2022.